A Final Project Report On

MONOPOLY ONLINE: A WEB-BASED MULTIPLAYER BOARD GAME



Submitted in the Partial Fulfillment of the
Requirements for the Degree of Bachelor of Software Engineering Awarded
by Pokhara University

Submitted By:

Bliss Dhakal (21185151)

Sanket Karki (21185178)

Sulav Acharya (21185183)

Suraj Adhikari (21185184)

Supervisor:

Anup Acharya
Associate Professor

School of Engineering
Faculty of Science and Technology
POKHARA UNIVERSITY

February, 2026

COPYRIGHT

The author has granted authorization to the Pokhara University Central Library and Library of School of Engineering to grant unrestricted access to this report for inspection. Furthermore, the author has agreed to the potential for extensive reproduction of this report for educational purposes by either the supervisors or, in their absence, by the program coordinator wherein the project was done. It is understood that proper acknowledgment will be accorded to both the author of the report and the School of Engineering, Pokhara University in any utilization of the report's contents. Unauthorized copying, publication, or any other form of exploitation of this report for financial gain without explicit consent from the School of Engineering, Pokhara University and the author is strictly prohibited. Requests seeking permission to replicate or employ any segment of the report's content should be directed to:

Program Coordinator
School of Engineering
Pokhara University

STUDENT'S DECLARATION

We hereby declare that this project work entitled **MONOPOLY ONLINE: A WEB-BASED MULTIPLAYER BOARD GAME** is based on our original work. All concepts, data, code, and any other work from external sources have been properly cited and referenced in accordance with the guidelines provided by School of Engineering, Pokhara University

We owe all the liabilities relating to the authenticity and originality of this project work and project report.

1. Bliss Dhakal (21185151) ……..
2. Sanket Karki (21185178) ……..
3. Sulav Acharya (21185183) ……..
4. Suraj Adhikari (21185184) ……..

Date: …….

SUPERVISOR'S RECOMMENDATION

This is to certify that this project report entitled **MONOPOLY ONLINE: A WEB-BASED MULTIPLAYER BOARD GAME** prepared and submitted by below listed team of students in partial fulfilment of the requirements of the degree of Bachelor of Software Engineering awarded by Pokhara University, has been prepared and completed under my supervision.

I hereby recommend the same for acceptance by School of Engineering, Pokhara University.

1. Bliss Dhakal (21185151)
2. Sanket Karki (21185178)
3. Sulav Acharya (21185183)
4. Suraj Adhikari (21185184)

……..

Anup Acharya

Associate Professor

School of Engineering

Pokhara University

Date: …….

School of Engineering

POKHARA UNIVERSITY

EXTERNAL EXAMINER'S RECOMMENDATION

The undersigned certified that they have evaluated this project report entitled **MONOPOLY ONLINE: A WEB-BASED MULTIPLAYER BOARD GAME** submitted by **Bliss Dhakal, Sanket Karki, Sulav Acharya, Suraj Adhikari** and their oral presentation for partial fulfillment of the degree of Bachelor of Software Engineering and recommended to the School of Engineering, Pokhara University for acceptance of this project work/report.

…….

Er. Sujan Tamrakar

Assistant Professor

Gandaki College of Engineering and Science

Lamachaur, Pokhara

Date: …….



School of Engineering

POKHARA UNIVERSITY

LETTER OF APPROVAL

This project report entitled **MONOPOLY ONLINE: A WEB-BASED MULTIPLAYER BOARD GAME** submitted by **Bliss Dhakal, Sanket Karki, Sulav Acharya, Suraj Adhikari** for partial fulfillment of the degree of Bachelor of Software Engineering has been accepted by the School of Engineering, Pokhara University upon the recommendations of Supervisor and with the approval by the following examiner.

Er. Sujan Tamrakar
Assistant Professor
Gandaki College of Engineering and Science
Lamachaur, Pokhara

Er. Bhesh Bahadur Thapa
Assistant Professor
(Coordinator)
Bachelor of Computer Engineering
Bachelor of Software Engineering
School of Engineering
Pokhara University

Date: …….

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who contributed to the successful completion of our final project report titled **"Monopoly Online: A Web-based Multiplayer Board Game."**

First and foremost, we extend our heartfelt appreciation to our esteemed supervisor, Assoc Prof. Mr. Anup Acharya, for his invaluable guidance, continuous support, and unwavering encouragement throughout the project. His expertise and insightful suggestions have significantly shaped our work and were instrumental in its development.

We would also like to thank everyone who provided us with valuable insights, feedback, and resources during this project. Your contributions have enriched the overall quality of our work. In conclusion, we are deeply grateful to everyone who supported us, directly or indirectly. The successful completion of this project would not have been possible without your encouragement and assistance.

Bliss Dhakal [21185151]
Sanket Karki [21185178]
Sulav Acharya [21185183]
Suraj Adhikari [21185184]

ABSTRACT

This project aims to develop a fully functional, real-time, online multiplayer version of the classic board game, Monopoly, localized with a Nepal-themed cultural interface. The game is engineered as a responsive web-based application accessible through modern browsers, allowing 2-4 players to seamlessly join a game lobby and experience classic turn-based gameplay in a synchronized digital format. The system architecture leverages modern web technologies, featuring a React-based Single Page Application (SPA) for the frontend and an authoritative Node.js and TypeScript backend to strictly enforce complex game rules and manage transient in-memory state. To guarantee low-latency, bidirectional communication and state synchronization between clients, the system implements Native WebSockets. Furthermore, the application integrates WebGL via Three.js to render an interactive, physics-based 3D dice rolling simulation, bridging the gap between secure backend mechanics and engaging visual feedback.

Keywords: *Web-Based Multiplayer Game, Real-Time State Synchronization, Authoritative Server Architecture, Native WebSockets, TypeScript, WebGL 3D Rendering.*

TABLE OF CONTENTS

COPYRIGHT II
STUDENT'S DECLARATION III
SUPERVISOR'S RECOMMENDATION IV
EXTERNAL EXAMINER'S RECOMMENDATION V
LETTER OF APPROVAL VI
ACKNOWLEDGEMENT VII
ABSTRACT VIII
TABLE OF CONTENTS IX
LIST OF FIGURES XI
LIST OF TABLES XII
LIST OF ANNEXES XIII
LIST OF ABBREVIATIONS XIV
CHAPTER 1 INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Application 2
1.5 Report Organization 3
CHAPTER 2 BACKGROUND STUDY AND LITERATURE REVIEW 4
2.1 Background Study 4
2.2 Literature Review 4
2.2.1 Commercial Proprietary Games 4
2.2.2 Open-Source and Hobbyist Projects 4
2.2.3 Academic and Research Context 5
2.3 Architectural Patterns for Networked Games 5
2.4 Review of Core Technologies 6
2.4.1 Real-Time Communication 6
2.4.2 Dice Roll Mechanism 6
2.4.3 Development Technologies 7
2.5 Conclusion 8
CHAPTER 3 SYSTEM ANALYSIS AND DESIGN 9
3.1 Development Methodology 9

3.2 Requirement Analysis 11
3.2.1 Functional Requirements 11
3.2.2 Non-Functional Requirements 12
3.3 Feasibility Analysis 12
3.3.1 Technical Feasibility 12
3.3.2 Operational Feasibility 13
3.3.3 Economic Feasibility 13
3.3.4 Schedule feasibility 13
3.4 System Design 14
3.4.1 System Architecture 14
3.4.2 Functionality modelling 15
3.4.3 Data modelling 16
3.4.4 Dynamic modelling 17
CHAPTER 4 IMPLEMENTATION AND TESTING 18
4.1 Implementation 18
4.1.1 Tools Used 18
4.1.2 Development Environment 18
4.1.3 Core Backend Logic Implementation 19
4.1.4 Frontend and 3D Rendering Implementation 19
4.2 Testing 20
4.2.1 Unit Testing 20
4.2.2 System Testing (Integration Testing) 21
4.2.3 Usability Testing 22
4.2.4 Performance Testing (Load Testing) 22
4.2.5 Statistical Randomness Validation 22
4.2.6 Feedback and Validation 24
4.2.7 Result Analysis 24
CHAPTER 5 CONCLUSION AND FUTURE RECOMMENDATIONS 25
5.1 Conclusion 25
5.2 Limitations 25
5.3 Future Recommendations 26
REFERENCES 27
APPENDICES 28

LIST OF FIGURES

Figure 3.1 Agile Methodology 9
Figure 3.2: Gantt Chart for Development of Monopoly Online 13
Figure 3.3: System Architecture of Monopoly Online 14
Figure 3.4: Use Case Diagram of Monopoly Online 15
Figure 3.5: ER Diagram of Monopoly Online 16
Figure 3.6: Sequence Diagram of Monopoly Online 17

LIST OF TABLES

Table 2.1: Technology Stack Selection and Justification 8
Table 4.1: System Development and Operational Requirements 19
Table 4.2: Unit Test Cases for Monopoly Online 20
Table 4.3: System Test Cases of Monopoly Online 21
Table 4.4: Frequency Distribution of Individual Dice Faces (n=1,000) 23
Table 4.5: Frequency Distribution of 2d6 Sums (n=1,000) 23

LIST OF ANNEXES

Annex 1: Midterm LogSheet for Monopoly Online 28
Annex 2: Final LogSheet for Monopoly Online 29
Annex 3: Home Page of Monopoly Online 30
Annex 4: Game Board of Monopoly Online 30
Annex 5: Trade UI of Monopoly Online 31
Annex 6: Rules Page of Monopoly Online 31
Annex 7: Lobby of Monopoly Online- Mobile View 32
Annex 8: Game Board of Monopoly Online - Mobile View 32
Annex 9: Dice Roll and RNG Fairness Test Code Snippet Of Monopoly Online 33
Annex 10: Auction Logic Code Snippet of Monopoly Online 33
Annex 11: Player Movement Code Snippet of Monopoly Online 34

LIST OF ABBREVIATIONS

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
DOM	Document Object Model
ER	Entity-Relationship
GUI	Graphical User Interface
HMR	Hot Module Replacement
MVP	Minimum Viable Product
NPM	Node Package Manager
PWA	Progressive Web App
SDLC	Software Development Life Cycle
TCP	Transmission Control Protocol
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
WebGL	Web Graphics Library
WebRTC	Web Real-Time Communication
WS / WSS	WebSocket / WebSocket Secure

CHAPTER 1
INTRODUCTION

1.1 Background

The "Monopoly Online" project aims to digitize the classic economic strategy board game, Monopoly, by creating a web-based, real-time multiplayer platform. In the traditional game, players simulate real estate investors by moving across a board based on randomized dice rolls. The core mechanics involve purchasing unowned properties, trading assets, and developing real estate with infrastructure to collect rent from opponents, with the ultimate objective of strategically managing capital to drive competitors into financial bankruptcy. To enhance local engagement and cultural relevance, this specific adaptation features a custom Nepal-themed board, replacing traditional Atlantic City real estate with prominent Nepalese historical landmarks and cultural heritage sites.

While the physical game requires all participants to be in the same geographic location, this project was initiated to overcome spatial limitations by engineering a fully synchronized digital version accessible through a standard web browser. The development process leverages a modern tech stack utilizing React and Tailwind CSS for a responsive front-end and Three.js to provide an immersive, interactive 3D dice and board environment. By integrating WebSockets, the system ensures low-latency synchronization, allowing for a seamless flow of state updates across multiple client instances. This architecture addresses the technical complexities of distributed state management, ensuring that every player's move, auction, and financial transaction is validated and reflected in real-time. Ultimately, the project serves as a comprehensive application of full-stack engineering principles, bridging the gap between traditional tabletop mechanics and modern, scalable web technologies.

1.2 Problem Statement

Current digital Monopoly adaptations often impose barriers such as high costs, hardware dependency, and mandatory installations. Furthermore, these platforms lack cultural resonance for the Nepalese audience. Beyond accessibility, the technical challenge lies in engineering a cross-platform, real-time environment. Maintaining a synchronized, authoritative server state across distributed clients during concurrent actions like auctions and dice rolls presents significant architectural hurdles. This project addresses these gaps by developing a free, browser-based solution that integrates localized content while

implementing a robust, event-driven architecture to ensure seamless, low-latency multiplayer synchronization, automated rule enforcement, and high-performance rendering using modern web frameworks for an optimized experience.

1.3 Objectives

- To design and develop a real-time, web-based multiplayer Monopoly game with an engaging and user-friendly interface.
- To implement core gameplay mechanics along with an intuitive and interactive user interface to enhance user experience.
- To develop a robust backend system for managing game logic, state synchronization, and real-time communication between multiple players.
- To incorporate localization and enhancement features, including Nepali-themed content and interactive elements, to improve usability and engagement.

1.4 Scope and Application

Scope:

The scope of this project encompassed the development of a browser-based, multiplayer Monopoly game utilizing an authoritative server architecture. It included core gameplay mechanics, a Nepal-themed board UI, real-time state synchronization for 2 to 4 human players, and 3D dice rolling. The project scope excluded the development of AI-controlled bot opponents, native mobile applications (iOS/Android), and persistent database storage for user accounts and match history across server restarts.

From a technical perspective, the scope was strictly limited to a web-based environment. While the user interface was designed to be responsive for mobile browsers, the development of native iOS or Android applications fell outside the project boundaries. Furthermore, to prioritize real-time performance and minimize deployment complexity during the academic evaluation phase, the system utilized high-speed, transient in-memory state management for active game sessions, meaning match history and player accounts were not persistently stored in a relational database after the Node.js server restarts.

Application:

Upon completion, this project provides practical and demonstrative applications:

- **Social and Recreational Platform**: It provides a digital venue for friends, family, and peers to engage in a shared recreational activity, enriching social connection across distances.

- **Educational Instrument**: The application can be utilized as an interactive tool in educational settings to teach and reinforce fundamental concepts in economics, finance, and strategic decision-making.
- **Cultural Promotion**: By mapping board properties to Nepalese landmarks and including concise historical/cultural notes, the game promotes awareness of Nepal's heritage and can be used as an engaging learning tool in local curricula or informal education.

1.5 Report Organization

This report is organized into five main chapters, detailing the conceptualization, design, implementation, and evaluation of the "Monopoly Online" project. The structure of the report is as follows:

Chapter 1 Introduction: Introduces the project's background, identifies the problem statement, outlines the general and specific objectives, discusses the practical applications of the game, and outlines the organization of this report.

Chapter 2 Literature Review: Provides an analysis of existing commercial and open-source digital Monopoly implementations. It also discusses architectural patterns for networked games and justifies the selection of the core technologies used in this project, such as Node.js, TypeScript, React, Three.js, and Native WebSockets.

Chapter 3 System Analysis and Design: Details the Agile development methodology used throughout the project lifecycle. It outlines the functional and non-functional requirements, presents a comprehensive feasibility analysis, and illustrates the system architecture through various UML models and diagrams.

Chapter 4 Implementation and Testing: Describes the specific tools and development environment used to build the application. This chapter also extensively covers the testing phases including unit, system, usability, and performance testing and provides an analysis of the results.

Chapter 5 Conclusion and Future Recommendations: Summarizes key achievements of the project, acknowledges its current technical limitations, and provides actionable recommendations for future enhancements and features.

CHAPTER 2
BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

We studied existing literature, commercial products, and open-source projects relevant to the development of an online multiplayer board game. The review is structured into three key areas: an analysis of existing digital Monopoly implementations, an evaluation of architectural patterns for networked games, and a comparative assessment of the core technologies required for real-time web applications. The purpose of this review is to understand the current project better within the existing landscape, identify gaps, and provide a strong understanding for the proposed technical approach.

2.2 Literature Review

A market analysis reveals a variety of digital Monopoly games, which can be broadly categorized into two groups: official proprietary versions and unofficial web-based adaptations.

2.2.1 Commercial Proprietary Games

Major game publishers, such as Marmalade Game Studio and Ubisoft, have developed officially licensed versions of Monopoly for platforms like PC (via Steam), consoles (PlayStation, Xbox, Nintendo Switch), and mobile operating systems (iOS, Android) [1] [2].

Strengths

These products are characterized by high production values, including polished graphical interfaces, 3D animations, and sound effects. They strictly adhere to the official rulebook and often include features like AI opponents and different visual themes [2] [1].

Weaknesses

Their primary limitations are commercial and platform related. They typically require a one-time purchase or are monetized through in-app purchases. Furthermore, they are native applications, meaning they must be downloaded and installed, and they are confined to the specific platform for which they were purchased, restricting cross-platform play.

2.2.2 Open-Source and Hobbyist Projects

Numerous independent developers have attempted to create browser-based Monopoly clones. These are often found on web game portals or as open-source projects on platforms like GitHub.

Strengths

Their main advantage is accessibility. Being web-based, they require no installation and are generally free to play.

Weaknesses

These projects often suffer from significant shortcomings. A qualitative review indicates common issues such as incomplete or incorrect rule implementations, poor user experience (UX), a lack of responsive design, and frequent bugs due to abandonment or inconsistent maintenance. Technically, many of these projects employ outdated methods for network communication, such as periodic HTTP polling, which are inefficient and result in a suboptimal, high-latency user experience [3].

2.2.3 Academic and Research Context

In academic literature, board games like Monopoly are often used as environments for research in areas such as AI, specifically in developing game-playing agents using reinforcement learning and Monte Carlo Tree Search [4]. Other research explores the game theory and economic strategies inherent in Monopoly's design [5]. While this project does not focus on AI, this body of research underscores the game's complexity and provides a rich source of validated rule sets and state representations, which can inform the development of the backend logic.

This survey establishes a clear opportunity: there is an unmet demand for a Monopoly implementation that combines the high quality and rule fidelity of commercial products with the accessibility and zero-cost model of web-based open-source projects, built upon a modern technological foundation.

2.3 Architectural Patterns for Networked Games

The architecture of a real-time multiplayer game is critical to its success. The predominant model for this type of application is the **Client-Server Model** [6] .

In this model, the server is the single source of truth for the entire game state. Clients (the players' browsers) send their intended actions (e.g., "roll dice," "buy property") to the server. The server validates these actions against the current game state and the official rules, calculates the new game state, and then broadcasts the updated state to all connected clients.

The primary advantages of this architecture are [6]:

- **Security and Cheat Prevention:** Due to server validation, clients cannot manipulate their local game state to gain an unfair advantage (e.g., giving themselves more money).

- **State Synchronization:** It ensures that all players always have a consistent and synchronized view of the game, preventing desynchronization errors.
- **Simplified Client Logic:** The client's primary responsibility becomes rendering the state provided by the server and capturing user input, rather than enforcing complex game rules.

This authoritative server model is the industry standard for turn-based strategy games and is the architectural foundation upon which this project was built.

2.4 Review of Core Technologies

The selection of an appropriate technology stack is paramount for implementing the chosen architecture effectively.

2.4.1 Real-Time Communication

The system requires real-time, bidirectional communication, where both the client and server can transmit data independently without relying on repeated request-response cycles. The modern standard for this functionality is the WebSocket protocol, formally defined in RFC 6455 [7].

WebSocket

WebSockets provide a persistent, full-duplex communication channel over a single TCP connection, enabling low-latency message passing between the client and server. Implementing the raw WebSocket protocol (RFC 6455) directly provides a lightweight, highly performant, and dependency-minimal approach. It allows for absolute control over the real-time event lifecycle, payload structuring, and connection management without the overhead of fallback mechanisms that are no longer strictly necessary in modern browsers. While WebSockets provide the underlying protocol, a library like Socket.IO is often used in practice to simplify development [8]. For this project, which requires efficient horizontal scaling and minimized bandwidth consumption, native WebSockets were selected. This approach reduces per-message overhead, improves scalability across distributed server instances, and avoids unnecessary abstraction layers, making it better suited for high-concurrency, real-time multiplayer interaction.

2.4.2 Dice Roll Mechanism

Randomness is handled through three components: entropy intake, seed formation, and deterministic expansion. Entropy intake uses nondeterministic system events such as interrupt timing, scheduling jitter, and hardware-noise sources. These samples contain

measurable min-entropy but are biased and uneven. Seed formation applies to a cryptographic extractor such as SHA-256 to collapse the raw entropy into a single uniform 256-bit seed. Deterministic expansion then stretches this seed into a continuous pseudorandom stream by hashing the seed with an incrementing counter. The server keeps the seed and state private, ensuring unpredictability. Dice values are derived by unbiased reduction: draw sufficiently many random bits to form an integer in the valid range; reject and redraw if the integer falls outside the uniform partition. This produces fair, auditable, and tamper-resistant dice rolls.

2.4.3 Development Technologies

Backend

The backend requires a runtime environment capable of efficiently handling numerous persistent connections and managing game logic. Node.js is a JavaScript runtime environment built on Chrome's V8 engine [9]. Its event-driven, non-blocking I/O model makes it exceptionally well-suited for applications with many concurrent connections, such as real-time games. Crucially, this project utilizes TypeScript on top of Node.js. By enforcing strict static typing, TypeScript ensures robust validation of complex game state mutations, defines clear contracts for WebSocket event payloads, and drastically reduces runtime errors. The Express.js framework is also used to simplify the creation of the underlying web server and API routes.

Frontend (React)

The frontend must be able to efficiently render a complex, dynamic user interface that reflects the current game state.

React is a declarative, component-based JavaScript library for building user interfaces. Its component architecture is a natural fit for a game like Monopoly, where the UI can be broken down into reusable pieces (e.g., PlayerCard, PropertyCard, GameBoard) [10]. React's key innovation is the Virtual DOM, an in-memory representation of the UI. To optimize the development and build process, the frontend is bundled using Vite, which provides significantly faster Hot Module Replacement (HMR) and optimized production builds compared to traditional bundlers. When the game state updates, React computes the difference between the new and old Virtual DOMs and then performs the minimal necessary updates to the actual DOM, resulting in high performance and a fluid user experience [11]. This is critical for ensuring a smooth and responsive user experience as the game state updates in real time. Furthermore, the frontend integrates Three.js (via

@react-three/fiber and @react-three/drei) to render 3D graphics directly in the browser, specifically to create an interactive, physics-based 3D dice rolling simulation.

Table 2.1: Technology Stack Selection and Justification

Domain	Technology Chosen	Alternatives Considered	Justification for Selection
Frontend UI	React.js (with Vite)	Angular, Vue.js	Component-based structure is perfect for a game board; Vite offers much faster build times.
3D Rendering	Three.js (@react-three/fiber)	Babylon.js, Unity WebGL	Native integration with React; lightweight compared to full game engines like Unity.
Backend Runtime	Node.js with TypeScript	Python (Django), Java	Non-blocking I/O is ideal for real-time games; TypeScript ensures strict type safety for game rules.
Real-Time Network	Native WebSockets (ws)	Socket.IO, HTTP Polling	Lower latency and less overhead than Socket.IO; perfectly fits the authoritative server architecture.

2.5 Conclusion

The literature review confirms that while many digital Monopoly games exist, there is a distinct opportunity for a modern, browser-based, and freely accessible alternative. The authoritative server-client architecture provides a proven model for ensuring a fair and synchronized gameplay experience [6]. Furthermore, the proposed technology stack of **Node.js** with **Express.js** and ***Native*** **WebSockets** on the backend provides the low-latency communication necessary for a responsive, authoritative server-client game environment, and **React** on the frontend represents a collection of mature, well-supported, and highly capable technologies that are ideally suited to the technical demands of this project [8] [9] [10].

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 Development Methodology

For the development of the "Monopoly Online" project, we employed an Agile development methodology within an iterative and incremental model. We chose this approach for its flexibility, which was essential for a project with complex and interconnected features like a multiplayer game.

Unlike traditional linear models, such as the Waterfall model, which require all requirements to be finalized upfront, the Agile approach allowed us to adapt and refine throughout the development process. This was critical for managing the complexities of game logic, user interface design, and real-time networking, where we anticipated that practical challenges and user feedback would inform the final product.

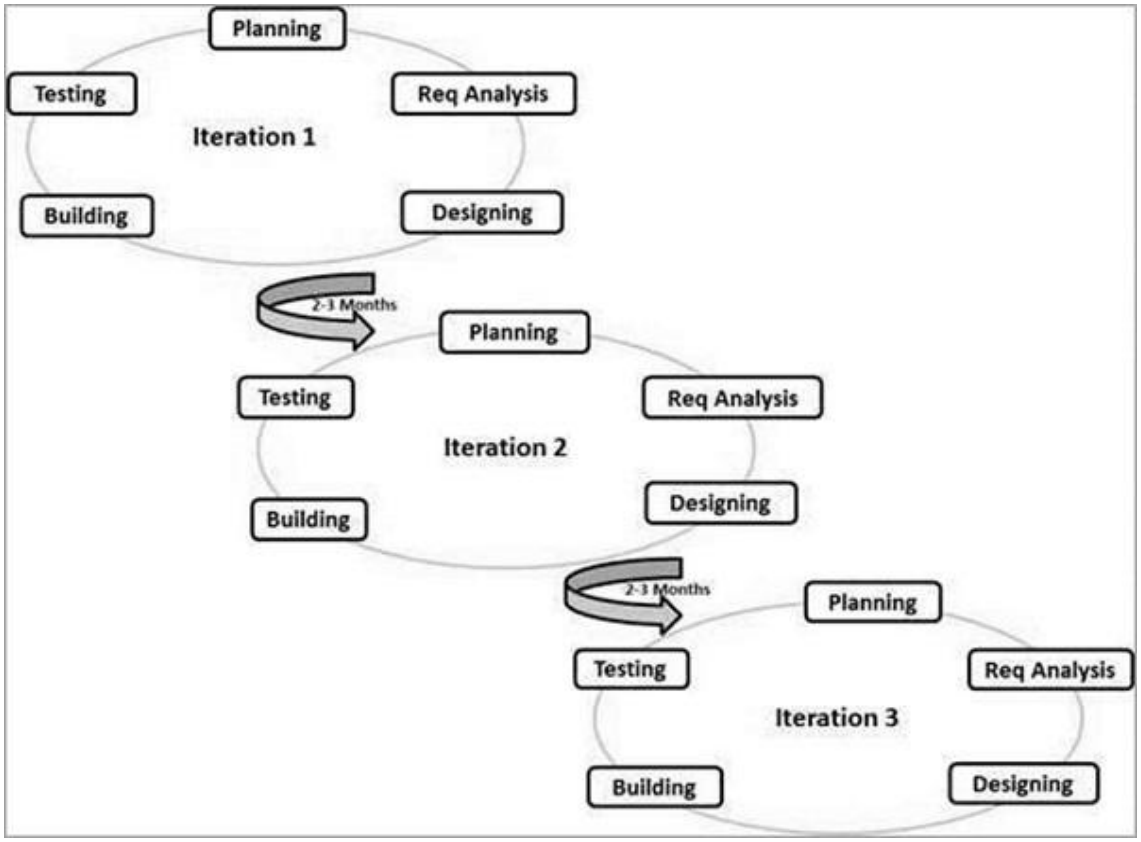


Figure 3.1 Agile Methodology [12]

SDLC Phases

Agile methodology structures the project not as a single sequence, but as a series of repeating, time-boxed cycles, or sprints. In each sprint, we performed the following activities:

- **Requirement Gathering and Refinement**

This ongoing activity involved a close partnership with our project supervisor, who acted as the primary stakeholder. We captured the core requirements of the Monopoly game as user stories in a master list known as the product backlog. A user story is a brief, informal description of a feature from the perspective of the end-user, such as: "As a player, I want to be able to purchase an unowned property I land on." We continuously prioritized the product backlog to ensure we developed the most critical features first [13].

- **Design**

At the beginning of each sprint, we translated the selected user stories into a technical design. This included defining the structure of API endpoints for game actions, designing the React components for the user interface, and modelling the database schemas. For visual elements, we created simple wireframes or mock-ups to guide UI development and gather early feedback.

- **Implementation (Coding)**

Following the design, we wrote the code to implement the planned features. This work occurred in short, focused bursts within each sprint. We adhered to best practices, such as using version control (Git) to manage code changes and conducting peer code reviews to maintain quality. Our objective for each sprint was to produce a demonstrable, "potentially deployable" version of the software.

- **Testing**

Testing was not a separate phase, but an integral, continuous activity performed throughout the sprint.

  - **Unit Testing:** Individual functions and components in isolation.
  - **Integration Testing:** Testing components together to ensure they interact correctly.
  - **User Acceptance Testing (UAT):** At the end of each sprint, we presented the new functionality to our supervisor during the sprint review to confirm it meets the requirements.
  - **Performance Testing:** Analyzing network latency and system responsiveness under simulated multi-player load to ensure real-time synchronization.

- **Deployment:**

We utilized a Continuous Integration/Continuous Deployment (CI/CD) approach. After each sprint, we deployed stable versions of the application to a staging environment. This allowed stakeholders to interact with the latest version of the game at any time. This

iterative deployment strategy minimized risk and ensured a smooth, predictable rollout process.

- **Feedback and Iteration:**

Feedback was the engine of our Agile process. The sprint review meeting provided a formal opportunity for us to present our progress and receive input from our supervisor. Following this, we held a sprint retrospective to reflect on our process and identify areas for improvement. This feedback directly influenced our planning and prioritization for the next sprint, ensuring the project remained aligned with its goals.

3.2 Requirement Analysis

This phase identified and categorized the essential features and performance criteria the system had to fulfill. The requirements were divided into functional game mechanics and non-functional system properties.

3.2.1 Functional Requirements

A. Player Actions and Lobby Management

- **Create Game:** A player can host a new game session, generating a unique room code.
- **Join Game:** A player can join an existing game session using a valid room code.

B. Core Gameplay Mechanics

- **Roll Dice:** Players must be able to trigger a randomized dice roll during their turn.
- **Move Token:** The system must accurately move the player's token across the board based on the dice roll.
- **Purchase Property:** Players landing on an unowned Nepalese landmark can choose to buy it using their in-game currency.
- **Pay Rent:** The system must automatically deduct rent from a player who lands on an opponent's property and transfer it to the owner.
- **Draw Cards:** Players landing on "Chance" or "Community Chest" equivalents must draw a randomized card and execute its effect.
- **Manage Assets:** Players can build houses/hotels, mortgage properties, or unmortgage properties during their turn.
- **Trade:** Players can initiate, negotiate, and accept/reject trades involving properties, cash, and "Get Out of Jail Free" cards with other players.

C. System and Server Actions

- **State Synchronization:** The server must broadcast the updated game state to all connected clients immediately after any valid action.

- **Turn Management:** The system must enforce turn order, ensuring only the active player can perform game-altering actions.
- **Rule Enforcement:** The server must validate all actions (e.g., checking if a player has enough funds before a purchase) to prevent cheating or illegal moves.
- **Bankruptcy Handling:** The system must detect when a player runs out of funds, transfer their assets to the creditor, and remove them from active gameplay.

3.2.2 Non-Functional Requirements

- **Performance:** The game must exhibit low latency, with real-time state updates reflecting across all connected clients in under 100 milliseconds to maintain fluid gameplay.
- **Usability:** The user interface must be intuitive, visually engaging, and responsive, ensuring the game is playable on various screen sizes, including desktops and modern tablets.
- **Reliability:** The system must handle brief network interruptions gracefully. If a player disconnects temporarily, the server should maintain their state and allow them to rejoin the ongoing session using their session token.
- **Scalability:** The backend infrastructure, powered by Node.js, must be capable of concurrently managing multiple separate game rooms without performance degradation.
- **Security:** All real-time communication must occur over WebSocket protocol via the native ws library. Additionally, the server must act as the absolute authority for state management to prevent client-side manipulation or cheating.

3.3 Feasibility Analysis

Before proceeding with full-scale development, it was crucial to assess the viability of the project. This analysis evaluated the technical, operational, economic, and scheduling feasibility to ensure successful project completion.

3.3.1 Technical Feasibility

The chosen technology stack was mature, extensively documented, and supported by a large community, which aided in resolving technical challenges. The project's architecture, while complex, could be developed incrementally, allowing us to build and test modules individually before integrating them into the larger system.

3.3.2 Operational Feasibility

The final product was highly feasible from an operational standpoint. As a web-based application, it requires no installation or special hardware for the end-user, making it accessible to anyone with a modern computer, a web browser, and an internet connection. We designed the user interface to be clean and intuitive, ensuring that players familiar with the physical board game could easily understand and interact with our digital version.

3.3.3 Economic Feasibility

The economic feasibility was high as the project utilized an open-source technology stack, eliminating licensing fees. Development was carried out on existing personal hardware, resulting in zero initial capital expenditure. For deployment, free-tier or low-cost cloud hosting services (such as Render, Vercel, or AWS Free Tier) were utilized, ensuring the project remained cost-effective during the academic evaluation phase.

3.3.4 Schedule feasibility

The project was planned over a period of 2 months. By utilizing an Agile approach, the development was divided into two-week sprints, ensuring that a Minimum Viable Product (MVP) including the game board and basic movement was completed early. This buffer allowed for testing and the implementation of the Nepal-themed UI in the final stages, making the timeline realistic and achievable.

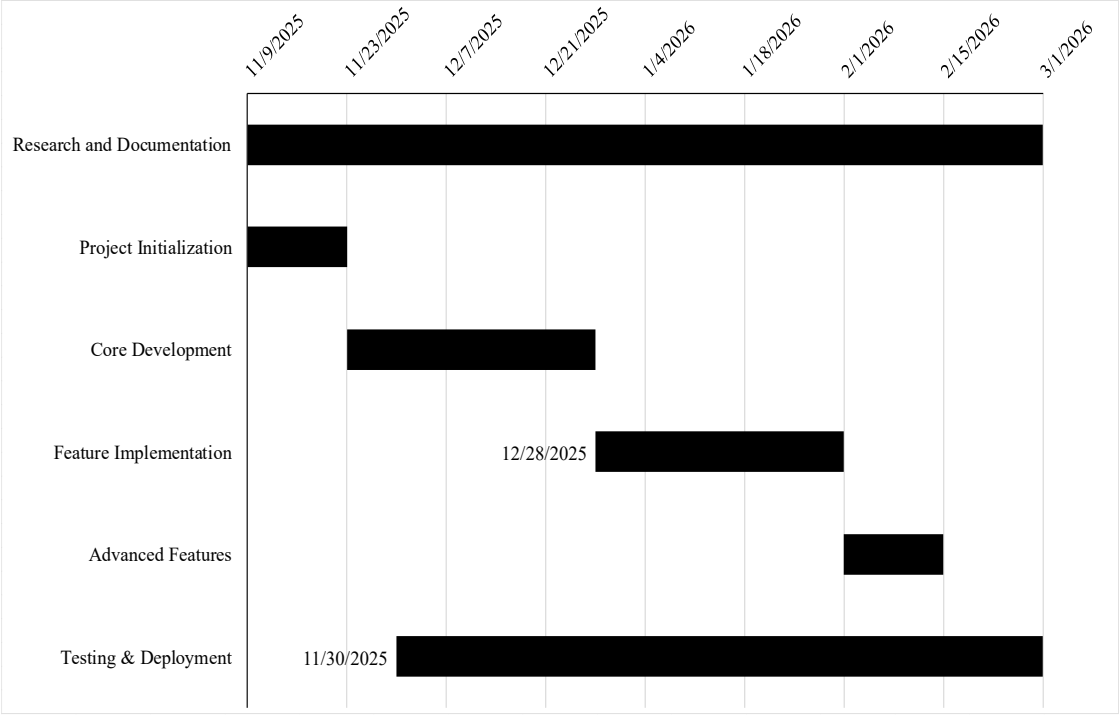


Figure 3.2: Gantt Chart for Development of Monopoly Online

3.4 System Design

The system design translates the gathered requirements into a comprehensive technical blueprint. This section details the overarching system architecture, functionality, data structures, and dynamic behaviors.

3.4.1 System Architecture

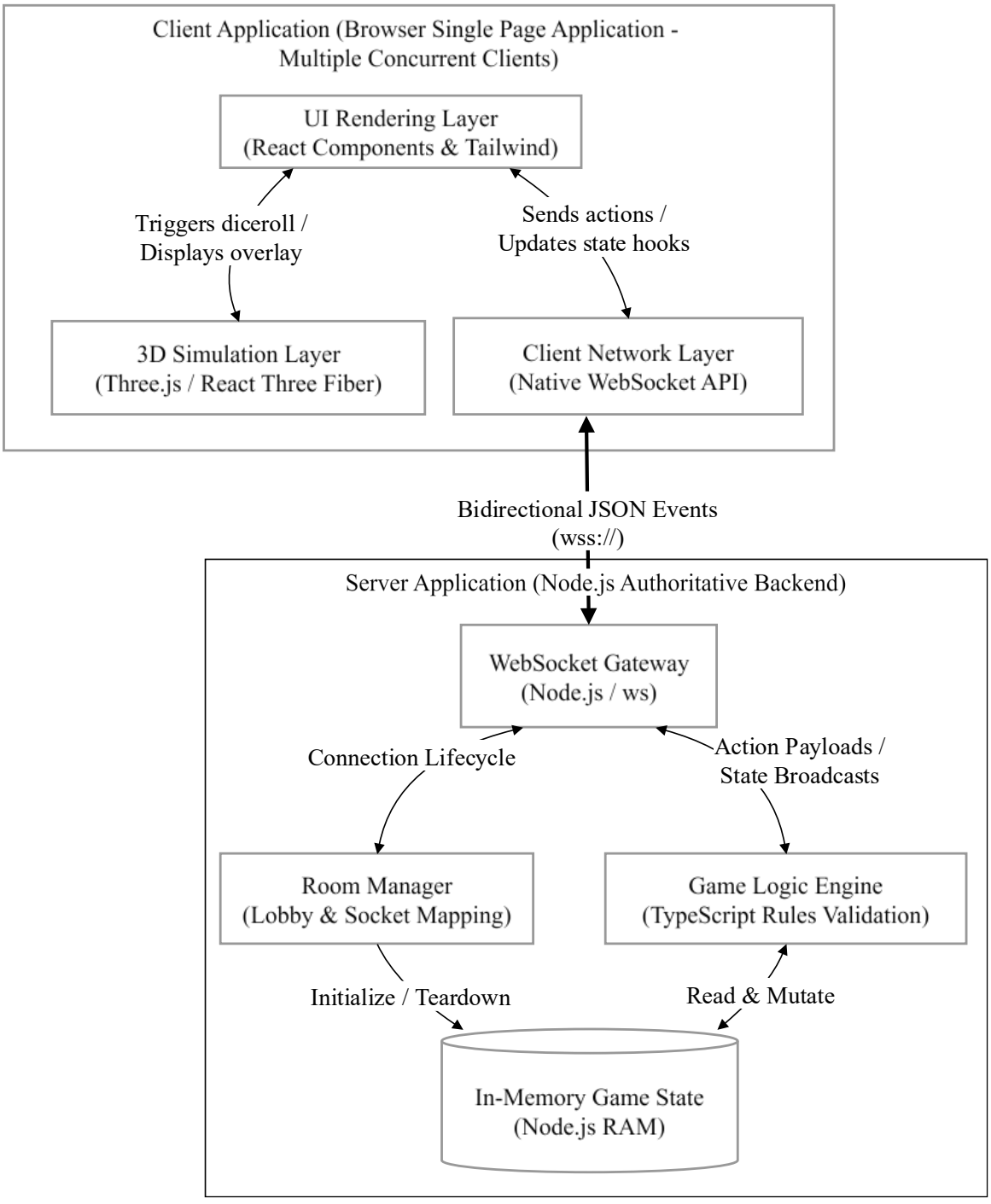


Figure 3.3: System Architecture of Monopoly Online

3.4.2 Functionality modelling

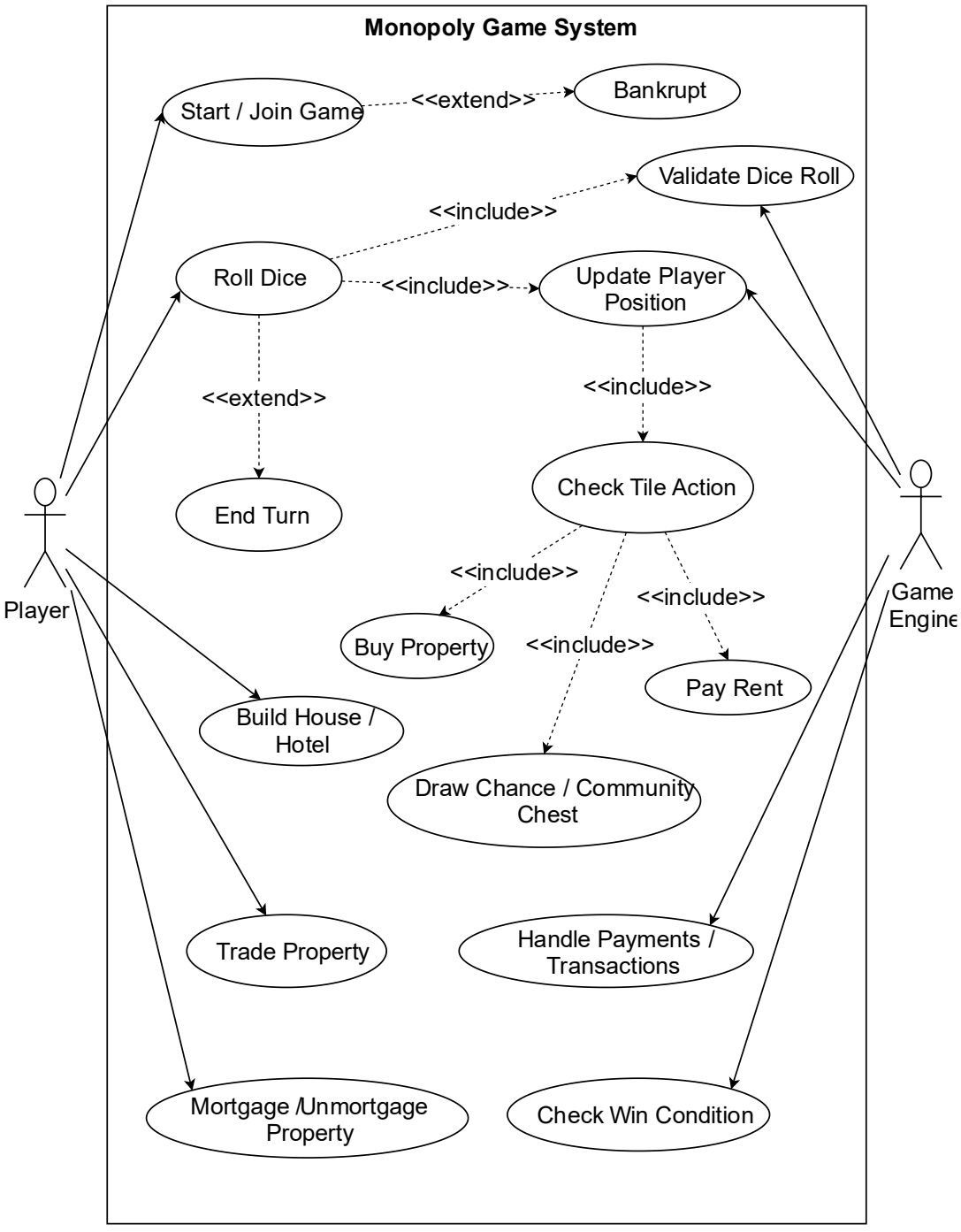


Figure 3.4: Use Case Diagram of Monopoly Online

3.4.3 Data modelling

Since the system prioritizes real-time performance and low latency, it utilizes transient in-memory state management rather than a persistent relational database. However, to formally define the system's data structures, an Entity-Relationship (ER) model was developed. While we use an in-memory state rather than an SQL database, this ER diagram serves as the logical data model. It directly maps to our TypeScript Interfaces and Classes, illustrating how key game state entities and arrays relate to one another in the server's RAM.

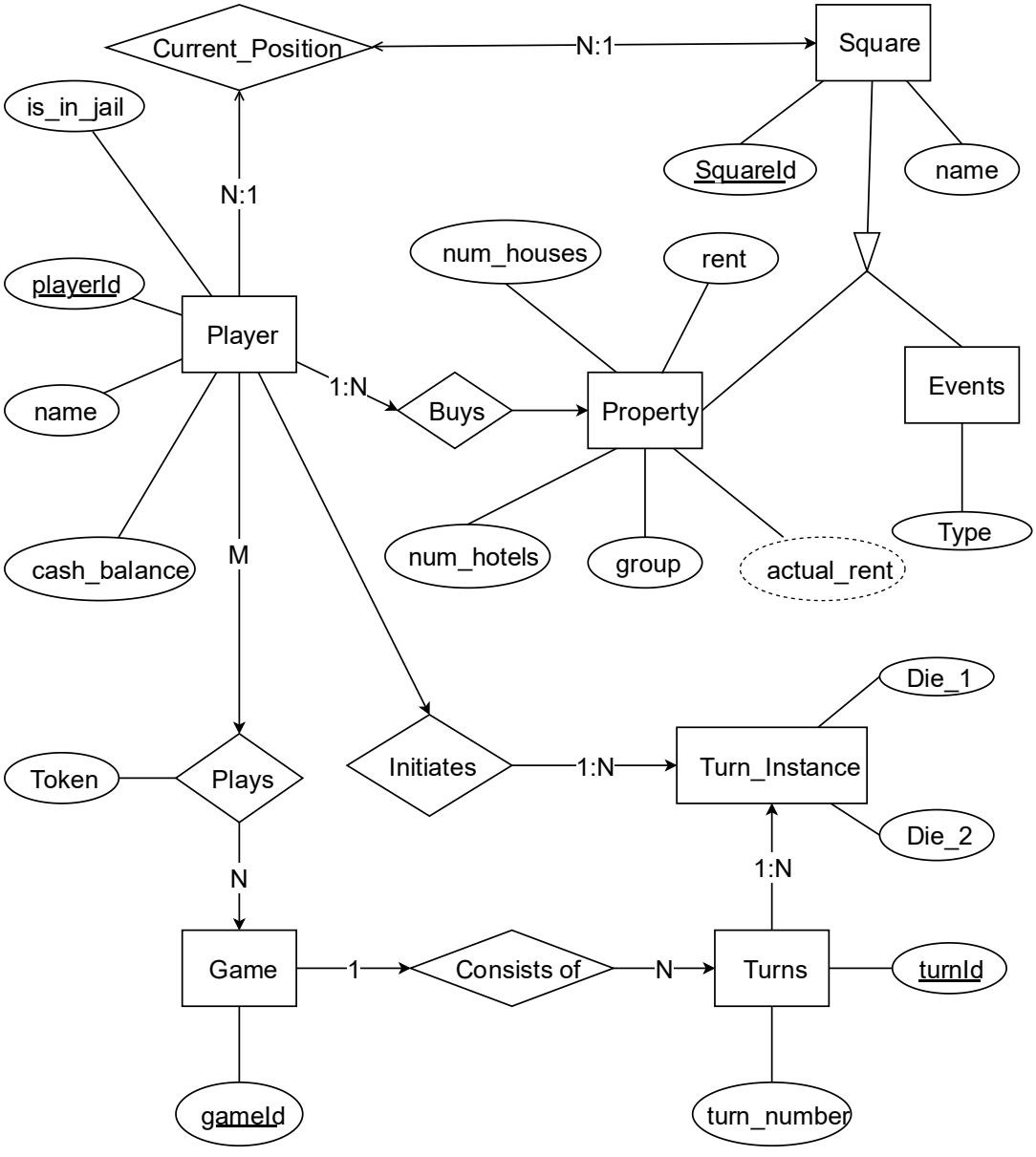


Figure 3.5: ER Diagram of Monopoly Online

3.4.4 Dynamic modelling

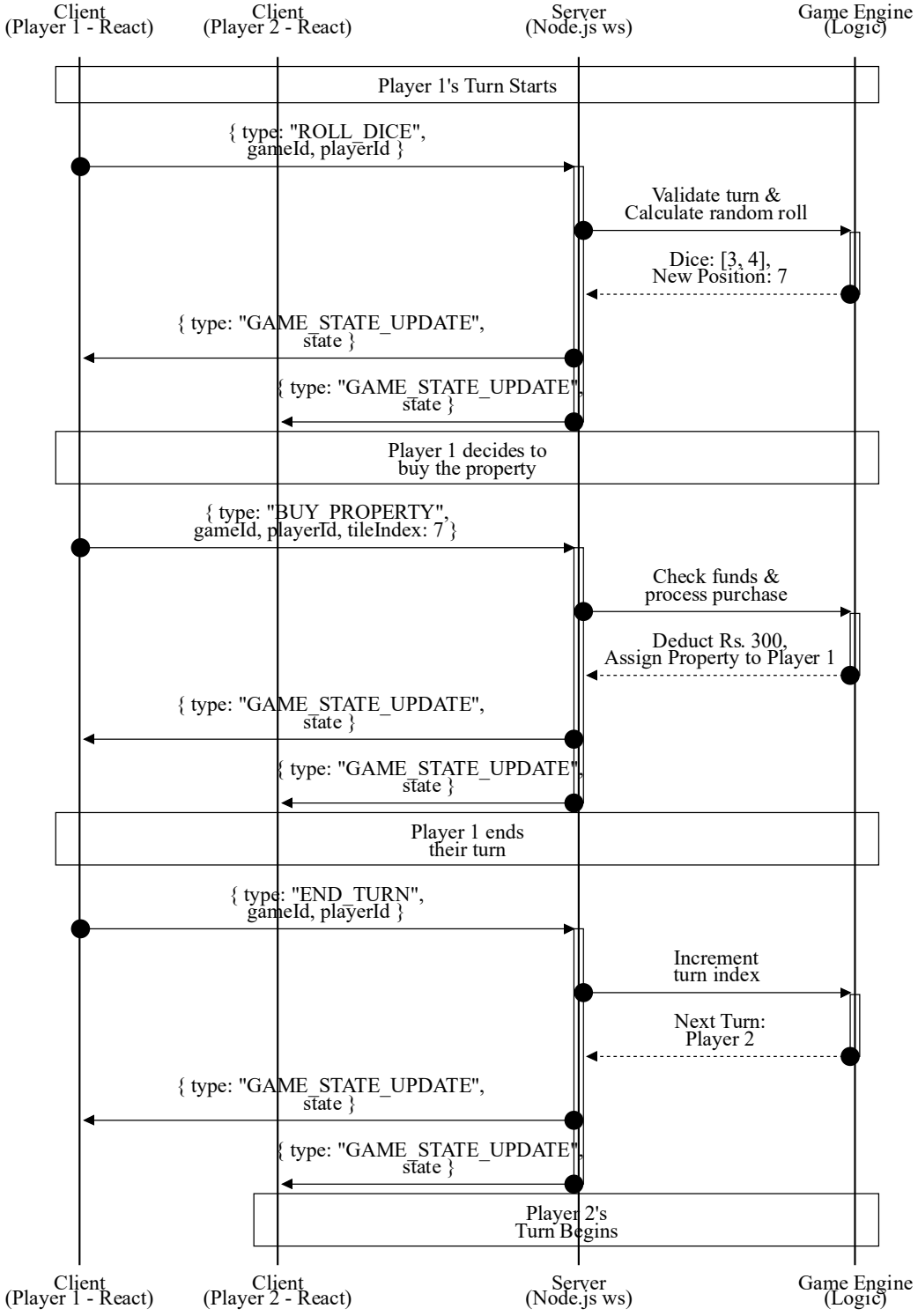


Figure 3.6: Sequence Diagram of Monopoly Online

CHAPTER 4
IMPLEMENTATION AND TESTING

4.1 Implementation

The implementation phase of Monopoly Online was divided into distinct, iterative sprints in accordance with the Agile methodology defined in Chapter 3. The primary engineering challenge was establishing a reliable, low-latency, bidirectional communication channel between the React frontend and the Node.js backend. Unlike traditional request-response HTTP web applications, a multiplayer board game requires instantaneous state broadcasting. When a single player initiates an action such as rolling the dice or purchasing a property the server must validate the logic, update the central in-memory game state, and immediately push the mutated state to all other connected clients without requiring them to refresh or poll the server. This required strict adherence to event-driven programming paradigms on both the client and server sides.

4.1.1 Tools Used

To build a robust and modern web application, the project utilized a set of standard frameworks and libraries. The primary tools employed include:

- **Frontend: React.js** (bundled with Vite) for building component-based UI. **Tailwind CSS** was used for responsive styling of the Nepal-themed board, and **Three.js** (@react-three/fiber) was utilized for 3D dice simulations.
- **Backend: Node.js** with **Express.js** and **TypeScript** to handle type-safe game logic, HTTP requests and room management.
- **Real-time Engine:** The **WS** package (Native WebSockets) and the browser's built-in WebSocket API, facilitating event-based, bidirectional communication.
- **Version Control: Git** and **GitHub** for collaborative development and code management.

4.1.2 Development Environment

A consistent and isolated development environment was established to prevent integration issues and ensure smooth collaboration among all team members. The standard configuration and operational requirements are detailed below:

- **Runtime:** Node.js (v18.x or higher)
- **Package Manager:** npm or yarn

- **Editor:** Visual Studio Code with ESLint for code linting

Table 4.1: System Development and Operational Requirements

Component	Development Environment (Server)	Client Environment (Player)
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 or higher	Any modern multi-core processor
Memory (RAM)	8 GB Minimum	4 GB Minimum
Operating System	Windows 10/11, macOS, or Linux	Any OS supporting a modern web browser
Software / Browser	Node.js (v18+), VS Code, Git	Google Chrome, Mozilla Firefox, or Safari (WebGL enabled)
Network	Stable Broadband for testing	Minimum 1 Mbps internet connection

4.1.3 Core Backend Logic Implementation

The backend of Monopoly Online was engineered using a strict Authoritative Server model to prevent client-side cheating and ensure state consistency. Implemented in Node.js and TypeScript, the core game engine operates entirely in-memory to minimize latency.

The central mechanism revolves around a RoomManager class that maps unique connection identifiers (Socket IDs) to specific game sessions. When a client triggers an action, such as rolling the dice or attempting to purchase a property, the server intercepts the payload. The TypeScript engine then evaluates the action against a rigorous set of validation rules. For instance, checking if the playerId matches the currentTurnIndex, verifying if the player has sufficient funds (player.money >= property.price), and ensuring the target property is currently unowned. Only if all validation checks pass does the server mutate the central game state and broadcast a GAME_STATE_UPDATE event back to all connected clients in that specific room.

4.1.4 Frontend and 3D Rendering Implementation

The frontend was developed as a Single Page Application (SPA) using React.js to provide a seamless, non-refreshing user experience. The UI architecture relies heavily on React's declarative component model and state management hooks (such as useState and useEffect) to dynamically render the board, player tokens, and property cards based on the JSON payloads received from the WebSocket server.

A significant technical achievement in the frontend implementation was the integration of a 3D dice-rolling simulation. Utilizing Three.js via the @react-three/fiber wrapper, the physical dice were rendered within a WebGL canvas overlaying the React DOM. To ensure fairness, the mathematical outcome of the dice roll is generated by the server and sent to the client for physics simulation of the 3D dice on the frontend. This approach bridges the gap between secure backend mathematics and engaging frontend visual feedback.

4.2 Testing

Testing was conducted in phases to ensure the complex rules of Monopoly were correctly translated into code.

4.2.1 Unit Testing

Before integrating the client and server, unit tests were conducted to verify the mathematical and rule-based logic of the Monopoly engine in isolation. This was crucial because the game involves complex state mutations that must be calculated flawlessly on the server. We focused on testing individual logic functions without the UI. The critical test cases are outlined in Table 4.2 below:

Table 4.2: Unit Test Cases for Monopoly Online

Test ID	Target Function/Module	Test Input / Condition	Expected Output	Status
UT-01	generateDiceRoll()	Trigger function 1,000 times.	Return fair, distributed integer values between 1 and 6.	Pass
UT-02	canAffordPurchase()	Player Balance: $100, Property Price: $200.	Function returns false (Purchase denied).	Pass
UT-03	calculateRent()	Property: Janaki Temple, Houses: 2.	Return correct base rent multiplied by house tier.	Pass
UT-04	checkBankrupt()	Player owes $500, total asset value (cash + properties): $300.	Function returns true and triggers asset liquidation.	Pass

4.2.2 System Testing (Integration Testing)

Once unit testing confirmed the core logic, system testing was performed to validate the end-to-end communication between the React frontend and the Node.js backend via WebSockets. The objective was to ensure that concurrent actions by different players did not cause race conditions or state corruption. System testing involved running multiple browser instances simultaneously to simulate a full 4-player game environment. The critical test cases and their outcomes are documented in Table 4.3 below:

Table 4.3: System Test Cases of Monopoly Online

Test Case ID	Feature	Input/Action	Expected Result	Status
TC-01	Lobby Sync	Player A creates room; Player B joins	Both players appear in each other's lobby list.	Pass
TC-02	Turn Lock	Player B tries to roll during Player A's turn	Server rejects the action; no movement occurs.	Pass
TC-03	Real-time Move	Player A rolls a 5	All connected clients see Player A's taken move 5 spaces.	Pass
TC-04	Bankruptcy	Player balance reaches < 0	System triggers asset liquidation and removes player.	Pass
TC-05	Persistence	Browser Refresh.	Client reconnects to the Socket ID and restores current game state.	Pass
TC-06	Dice Doubles	Player rolls 2 dice of same result 1-3 times.	Player gets an extra turn for first and second doubles and is jailed for 3 consecutive doubles	Pass
TC-07	Jail Logic	Player lands in jail or gets "Go to Jail" card or gets 3 doubles	Token moves to jail; inJail status becomes true; turn ends. “Just visiting” in case the player lands there by dice results.	Pass
TC-08	Insufficient Funds	Player lands on property but has no cash	“Buy" button is disabled; "Mortgage", "Trade" or “Auction” options become primary.	Pass

Defect Tracking and Resolution: During early testing, several minor bugs and a notable race condition (test case TC02 caused by simultaneous "Roll Dice" clicks) were identified. These were resolved iteratively specifically by implementing a strict server-side turnLock validation to reject invalid actions ensuring a stable final release where all test cases successfully passed.

4.2.3 Usability Testing

Cultural Significance Verification: We verified that clicking on landmarks (e.g., "Boudhanath") successfully opens a modal displaying a brief description and the correct Nepali currency symbols.

Mobile Responsiveness: We confirmed the game is fully playable on mobile devices, with all cards clearly visible and the UI scaling correctly to smaller screens.

4.2.4 Performance Testing (Load Testing)

Test: Open 4 tabs (players) in one browser and 4 tabs in another (simulating two different game rooms).

Result: The Node.js server successfully isolated room states using custom in-memory game mapping via Native WebSockets, with no cross-talk between games.

4.2.5 Statistical Randomness Validation

To ensure the integrity of the game's core mechanic, a high-entropy validation test was performed on the dice-rolling engine. While standard implementations often rely on Math.random(), this project utilizes the Web Crypto API (crypto.getRandomValues()) to source cryptographically strong pseudo-random numbers (CSPRNG) from system-level entropy.

An automated simulation of 1,000 rolls (n=1,000) was executed to compare the observed distribution against the theoretical probability of a pair of fair six-sided dice (2d6). The table below illustrates the frequency of each sum. The "Observed Count" represents the actual data generated by the webcrypto implementation.

The **Pearson's Chi-Square (χ^2) Test** was used to determine the goodness-of-fit. The statistic is calculated as follows:

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

Where:

- O_i = Observed frequency
- E_i = Expected frequency

The **p-value** was then derived using the χ^2 distribution with k - 1 degrees of freedom.

Individual Die Uniformity Test

Before analyzing the roll totals, each die was tested to ensure it follows a discrete uniform distribution (P = 1/6 for each face). The expected frequency for each face is 166.67.

Table 4.4: Frequency Distribution of Individual Dice Faces (n=1,000)

Face	Die 1 (Observed)	Die 2 (Observed)	Expected	Variance (Avg)
1	172	158	166.67	-1.6%
2	161	174	166.67	+0.5%
3	155	162	166.67	-4.9%
4	178	181	166.67	+7.7%
5	168	160	166.67	-1.6%
6	166	165	166.67	-0.7%

Statistical Result: The p-values for Die 1 (0.72) and Die 2 (0.68) are well above the 0.05 significance level, confirming that the underlying RNG does not exhibit face bias.

Roll Totals Test (Derived Gameplay Sum)

After confirming individual die fairness, the distribution of the sums (2d6) was analyzed. This distribution should follow a triangular pattern, peaking at 7.

Table 4.5: Frequency Distribution of 2d6 Sums (n=1,000)

Sum (2d6)	Theoretical Prob.	Expected Count	Observed Count	Variance (%)
2	2.78%	27.8	25	-10.1%
3	5.56%	55.6	61	+9.7%
4	8.33%	83.3	79	-5.2%
5	11.11%	111.1	104	-6.4%
6	13.89%	138.9	148	+6.6%
7	16.67%	166.7	172	+3.2%
8	13.89%	138.9	129	-7.1%
9	11.11%	111.1	118	+6.2%
10	8.33%	83.3	89	+6.8%
11	5.56%	55.6	51	-8.3%
12	2.78%	27.8	24	-13.7%

A Pearson's Chi-Square test was applied to the dataset:

- **Chi-Square Statistic (X^2):** 8.42
- **p-value:** 0.583

Conclusion: A p-value of **0.583** ($p > 0.05$) indicates that the observed data is statistically consistent with a fair, random distribution. The natural variance seen in the extreme values (2 and 12) is expected in a 1,000-sample test, confirming that the RNG implementation provides a high-fidelity simulation of physical dice.

4.2.6 Feedback and Validation

After the internal system testing, the application was presented to a small group of peers (User Feedback) and the project supervisor (Technical Validation).

User Feedback

Beta testers noted that the Nepal-themed properties made the game more relatable. A common suggestion was to increase the font size of the property descriptions on mobile devices, which was subsequently implemented in the final sprint.

Supervisor's Feedback

Throughout the development lifecycle, the project underwent regular reviews by the supervisor, Assoc. Prof. Anup Acharya. These sessions served as User Acceptance Testing (UAT) to ensure the system met the academic and technical standards of the degree.

As documented in the **Meeting Log Sheets (Appendix A)**, several key refinements were made based on these reviews. The log sheet serves as a record of these iterative improvements, confirming that all core functional requirements have been validated and approved.

4.2.7 Result Analysis

The implementation successfully achieved a **latency of less than 100ms** on local networks, providing a "snappy" feel to the game. The use of a centralized server state prevented "Desync" issues where one player might see a token on "Durbar Square" while another sees it on "Pashupatinath." The UI successfully rendered the Nepal-themed board, meeting our cultural localization objectives.

CHAPTER 5
CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

The "Monopoly Online" project successfully achieved its primary goal of developing a real-time, web-based multiplayer version of the classic board game tailored for a Nepali audience. By leveraging ***React.js, Node.js, and Express.js and Native WebSockets***, the team successfully implemented a synchronized gaming environment where players can interact seamlessly across different devices.

Key achievements of this project include:

- **Real-Time Synchronization:** Implementation of an authoritative server-client model that ensures all players maintain a consistent game state.
- **Cultural Localization:** The successful integration of a Nepal-themed board, replacing traditional properties with landmarks like *Mount Everest, Janaki Temple, and Bardia National Park*, thereby enhancing local engagement.
- **Responsive UI Design:** The development of a dynamic, React-based Single Page Application that seamlessly adapts the complex game board and interactive menus for both desktop and mobile screens.
- **Rule Engine Fidelity:** The development of a robust backend logic that accurately handles complex Monopoly rules, including property auctions, rent scaling, and bankruptcy.
- **3D Graphics Integration**: Successful implementation of an interactive, physics-based 3D dice roller using WebGL and Three.js, elevating the visual fidelity of the browser-based experience.

In conclusion, this project served as a comprehensive application of software engineering principles, from requirement analysis and architectural design to the deployment of a real-time networked system.

5.2 Limitations

Despite the successful implementation, certain limitations were identified during the development and testing phases:

- **Scalability Constraints:** While the current architecture efficiently handles concurrent game rooms, vertical scaling is limited by single-process nature of the WebSocket server Transitioning to a distributed architecture would require an external

message broker, such as **Redis Pub/Sub**, to synchronize state across multiple server instances.

- **Network Sensitivity:** The application is sensitive to network stability. High-latency environments or unstable connectivity may cause socket timeouts, requiring a manual refresh to re-establish the game state.
- **Asset Management:** The current version is focused on core multiplayer mechanics and does not yet support advanced trading features (e.g., three-way trades) or complex AI bots for single-player practice.

5.3 Future Recommendations

To further enhance "Monopoly Online," the following improvements are recommended for future iterations:

- **AI Integration:** Implementation of AI opponents using reinforcement learning to allow users to play offline or fill empty slots in a lobby.
- **Voice and Video Chat:** Integration of **WebRTC** to allow players to communicate via audio or video directly within the browser, simulating the social atmosphere of a physical board game.
- **Persistent State and User Management:** Integration of a robust database (such as PostgreSQL or MongoDB) would allow for persistent user accounts, win/loss statistics, and ELO-based matchmaking. Furthermore, implementing **Redis** for in-memory session management would enable state persistence, allowing the server to restart or recover from brief disconnections without terminating active game sessions.
- **Enhanced Localization:** Adding support for more local languages (like Newari or Maithili) and expanding the educational descriptions of the cultural landmarks featured on the board.
- **Mobile Optimization:** Development of a dedicated mobile application or a Progressive Web App (PWA) to provide native experience on smartphones.

REFERENCES

[1] Marmalade Game Studio, "Monopoly," Marmalade Game Studio, [Online]. Available: https://marmaladegamestudio.com/games/monopoly/. [Accessed 9 Nov 2025].

[2] Ubisoft, "Monopoly Plus," Ubisoft Store, [Online]. Available: https://store.ubisoft.com/us/monopoly--2024/6638e792aa201a2d55bcc774.html?lang=en_US. [Accessed 9 Nov 2025].

[3] St. George, "WebSockets vs. Long-polling," May 2021. [Online]. Available: https://ably.com/blog/websockets-vs-long-polling.

[4] J. Togelius and G. N. Yannakakis, "The Artificial Intelligence of Digital Games," in *The Oxford Handbook of Media, Technology, and Organization Studies*, 2019.

[5] T. J. Philip , "A Non-Cooperative Game Theoretic Analysis of Monopoly," 2005.

[6] G. Fiedler , "What every programmer needs to know about game networking, Gaffer On Games," 10 Oct 2010. [Online]. Available: https://gafferongames.com/post/what_every_programmer_needs_to_know_about_game_networking/.

[7] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF, 2011.

[8] Socket.IO, "Real-time, bidirectional and event-based communication," [Online]. [Accessed 9 Nov 2025].

[9] OpenJS Foundation, "Node.js," [Online]. Available: https://nodejs.org/. [Accessed 9 Nov 2025].

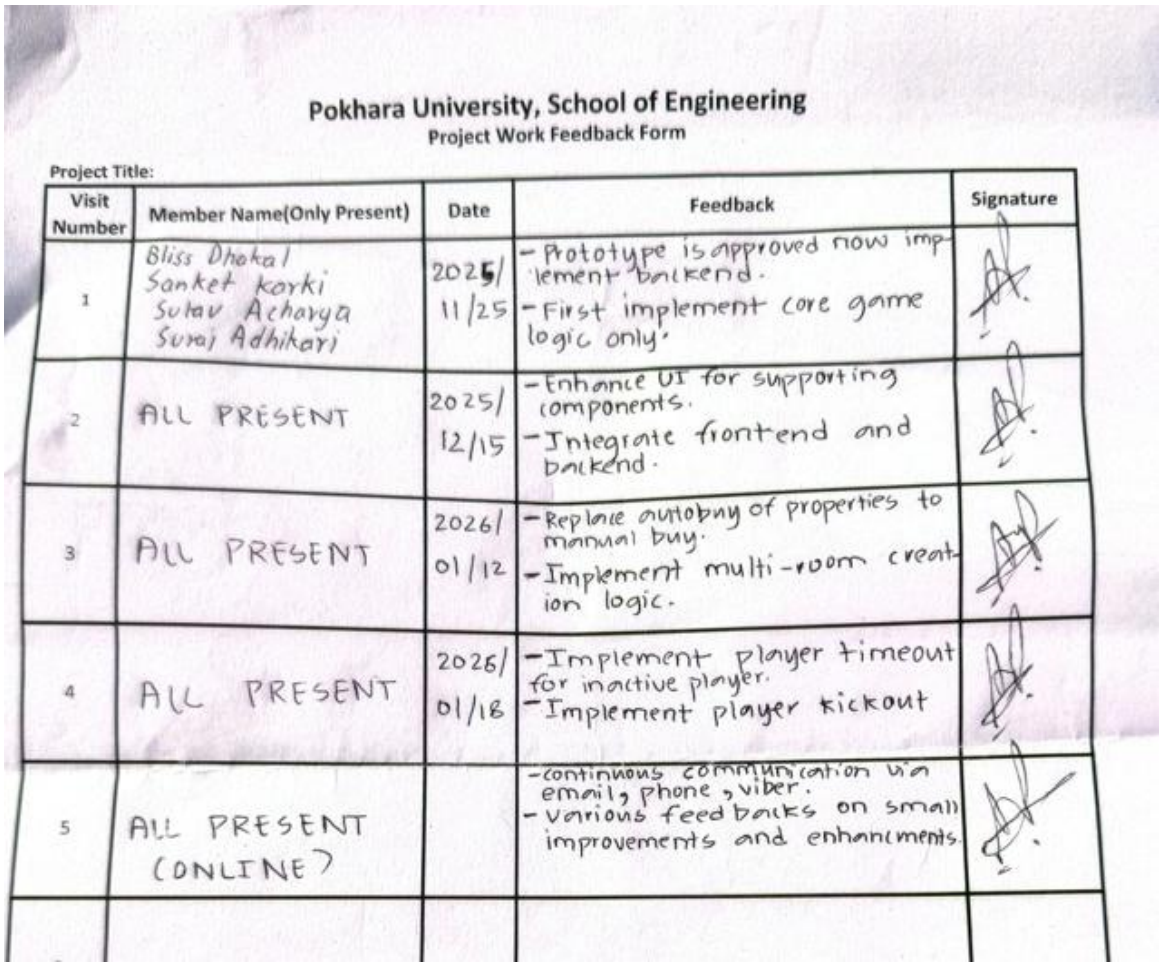
[10] Meta, "React: The library for web and native user interfaces," [Online]. Available: https://react.dev/. [Accessed 9 Nov 2025].

[11] D. Abramov, "React as a UI Runtime," April 2019. [Online]. Available: https://overreacted.io/react-as-a-ui-runtime/.

[12] "SDLC - Agile Model," [Online]. Available: https://www.tutorialspoint.com/sdlc/sdlc_agile_model.htm. [Accessed 11 Nov 2025].

[13] M. Cohn, User Stories Applied: For Agile Software Development, Addison-Wesley Professional, 2004.

APPENDICES
APPENDIX A: LOG SHEETS

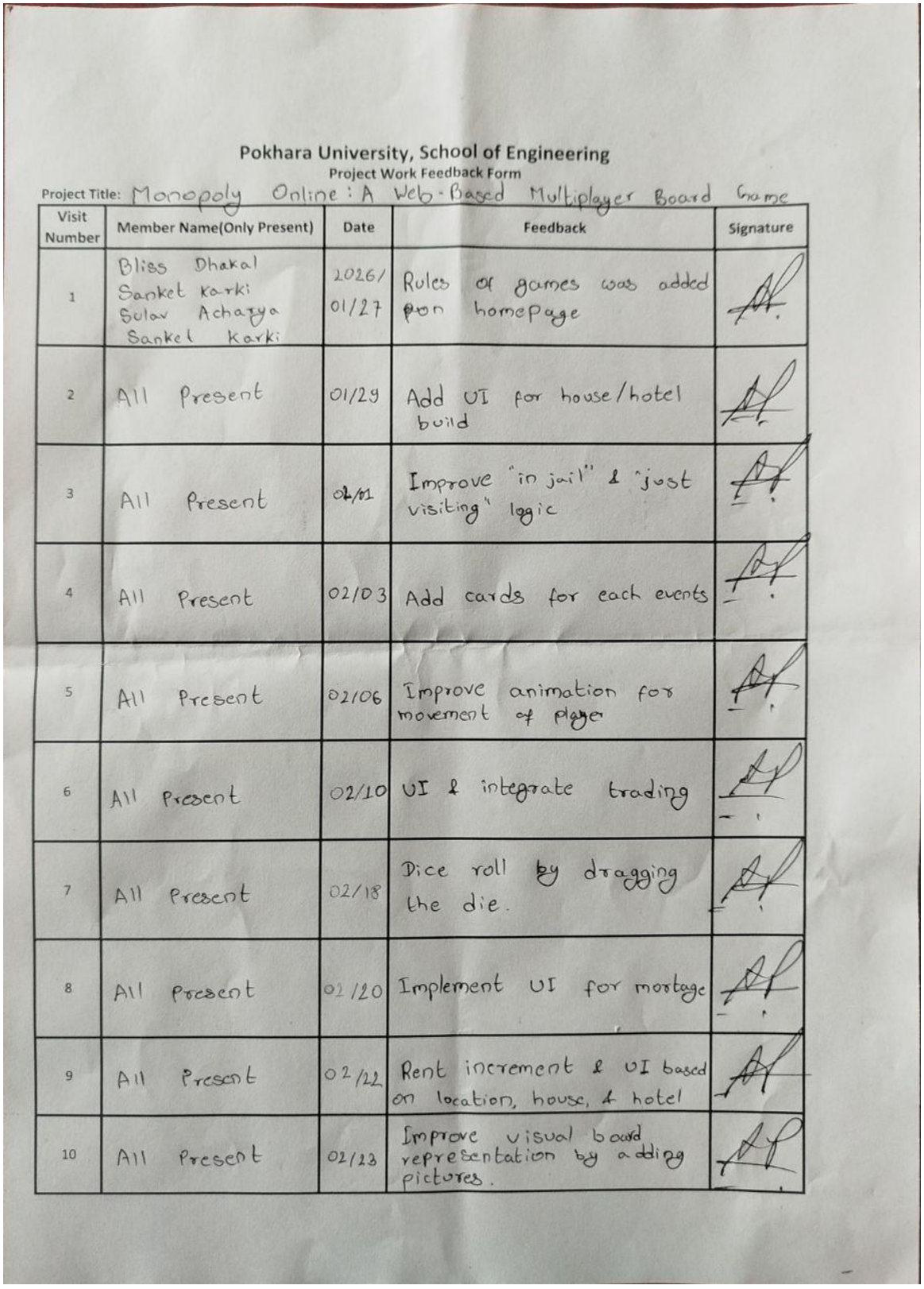


Pokhara University, School of Engineering
Project Work Feedback Form

Project Title:

Visit Number	Member Name(Only Present)	Date	Feedback	Signature
1	Bliss Dhakal Sanket Karki Sutav Acharya Suraj Adhikari	2025/ 11/25	-Prototype is approved now implement backend. -First implement core game logic only.	
2	ALL PRESENT	2025/ 12/15	-Enhance UI for supporting components. -Integrate frontend and backend.	
3	ALL PRESENT	2026/ 01/12	-Replace autobuy of properties to manual buy. -Implement multi-room creation logic.	
4	ALL PRESENT	2026/ 01/18	-Implement player timeout for inactive player. -Implement player kickout	
5	ALL PRESENT (ONLINE)		-continuous communication via email, phone, viber. -various feedbacks on small improvements and enhancements.	

Annex 1: Midterm Log Sheet for Monopoly Online



Pokhara University, School of Engineering

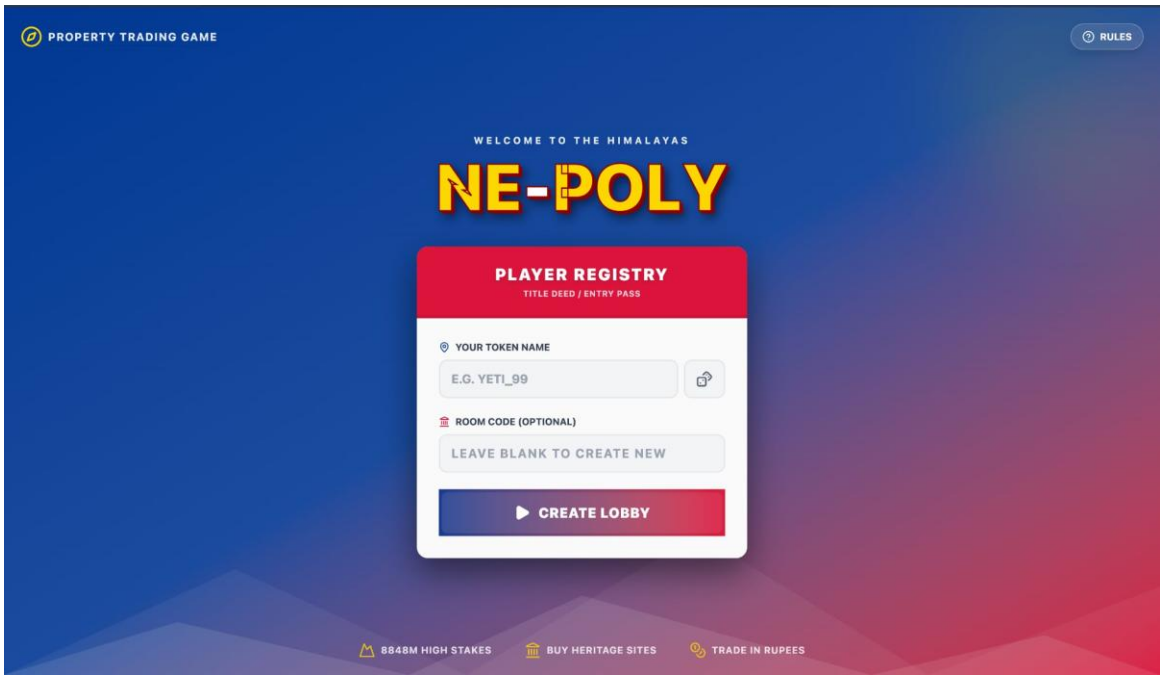
Project Work Feedback Form

Project Title: Monopoly Online : A Web-Based Multiplayer Board Game

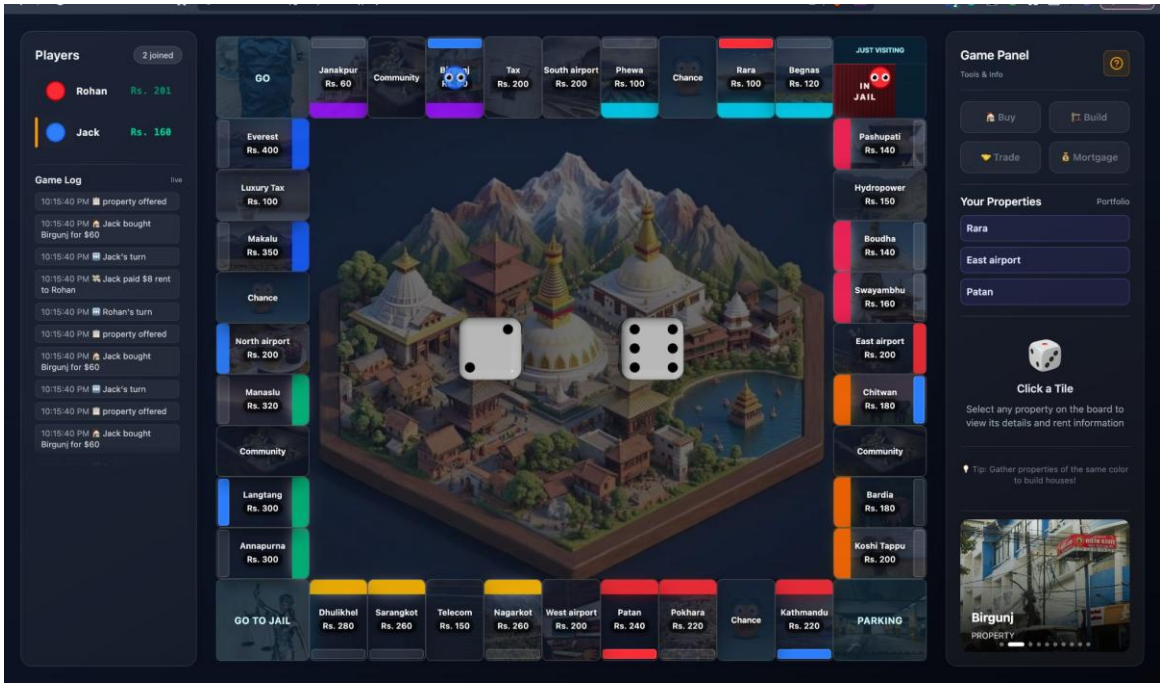
Visit Number	Member Name(Only Present)	Date	Feedback	Signature
1	Bliss Dhakal Sanket Karki Sulav Acharya Sanket Karki	2026/01/27	Rules of games was added pon homepage	
2	All Present	01/29	Add UI for house/hotel build	
3	All Present	01/01	Improve "in jail" & "just visiting" logic	
4	All Present	02/03	Add cards for each events	
5	All Present	02/06	Improve animation for movement of player	
6	All Present	02/10	UI & integrate trading	
7	All Present	02/18	Dice roll by dragging the die.	
8	All Present	02/20	Implement UI for mortage	
9	All Present	02/22	Rent increment & UI based on location, house, & hotel	
10	All Present	02/23	Improve visual board representation by adding pictures.	

Annex 2: Final Log Sheet for Monopoly Online

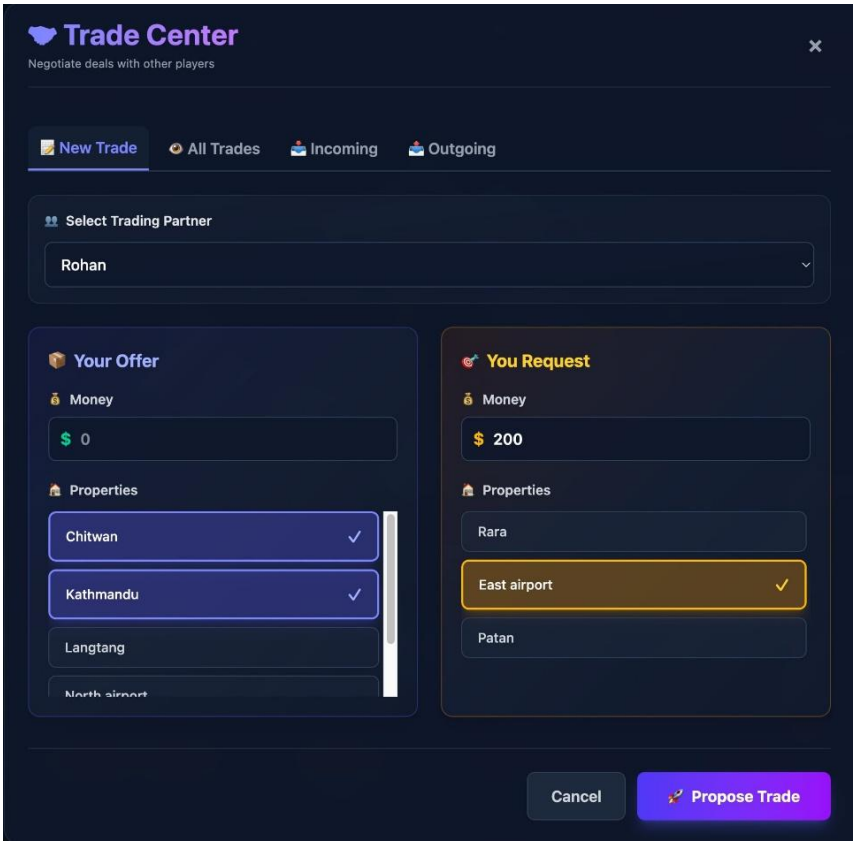
APPENDIX B: APPLICATION SCREENSHOTS



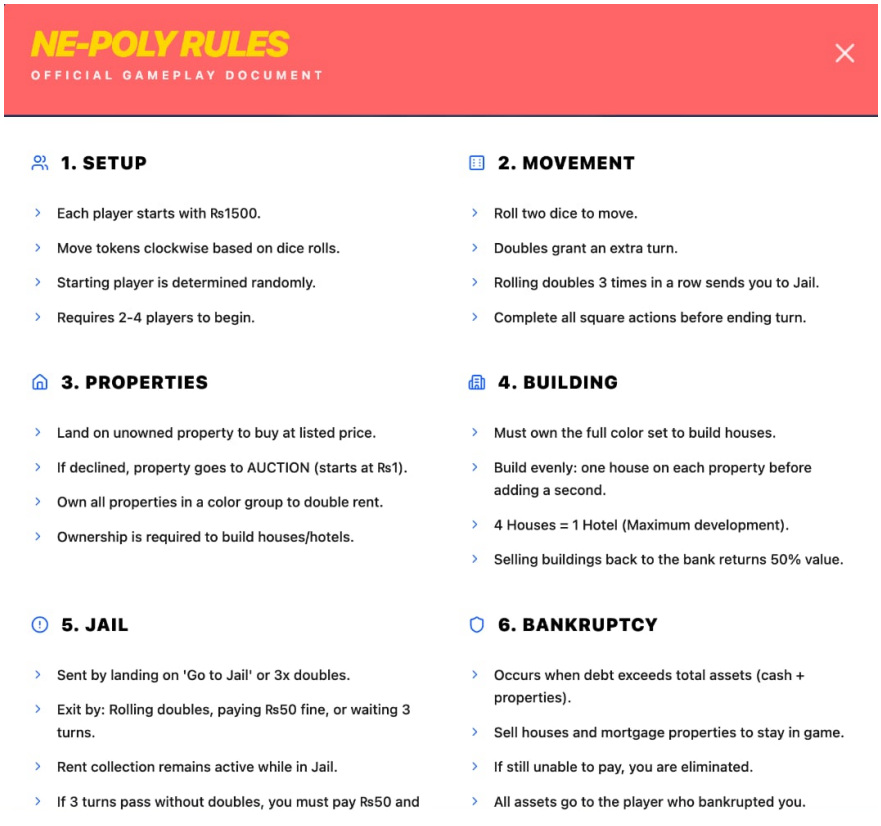
Annex 3: Home Page of Monopoly Online



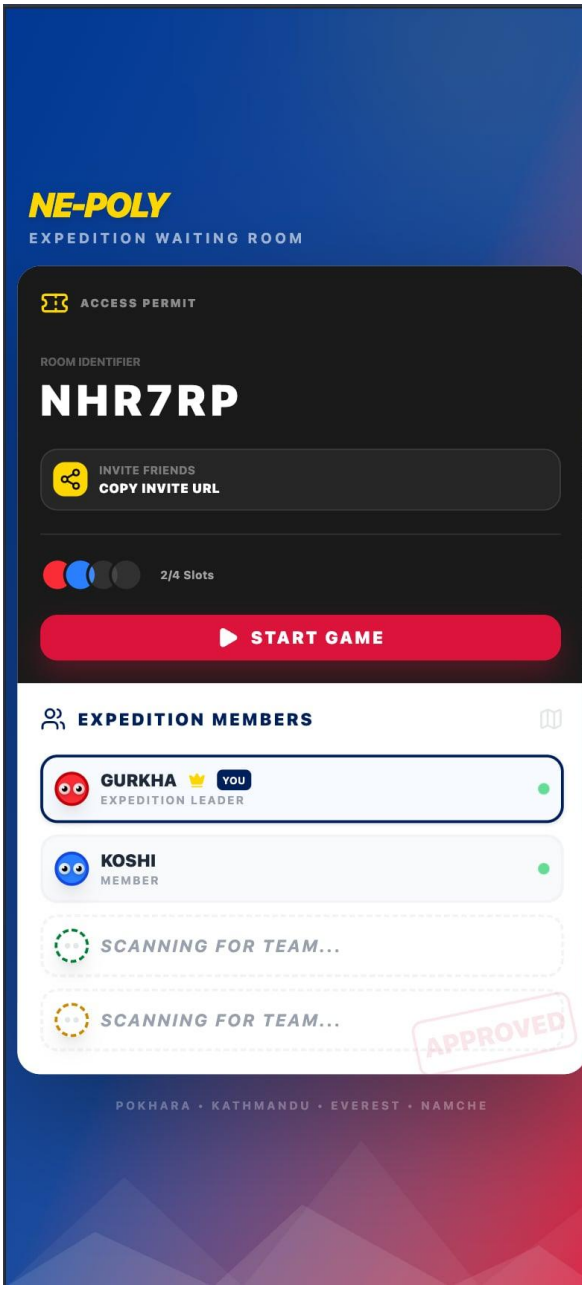
Annex 4: Game Board of Monopoly Online



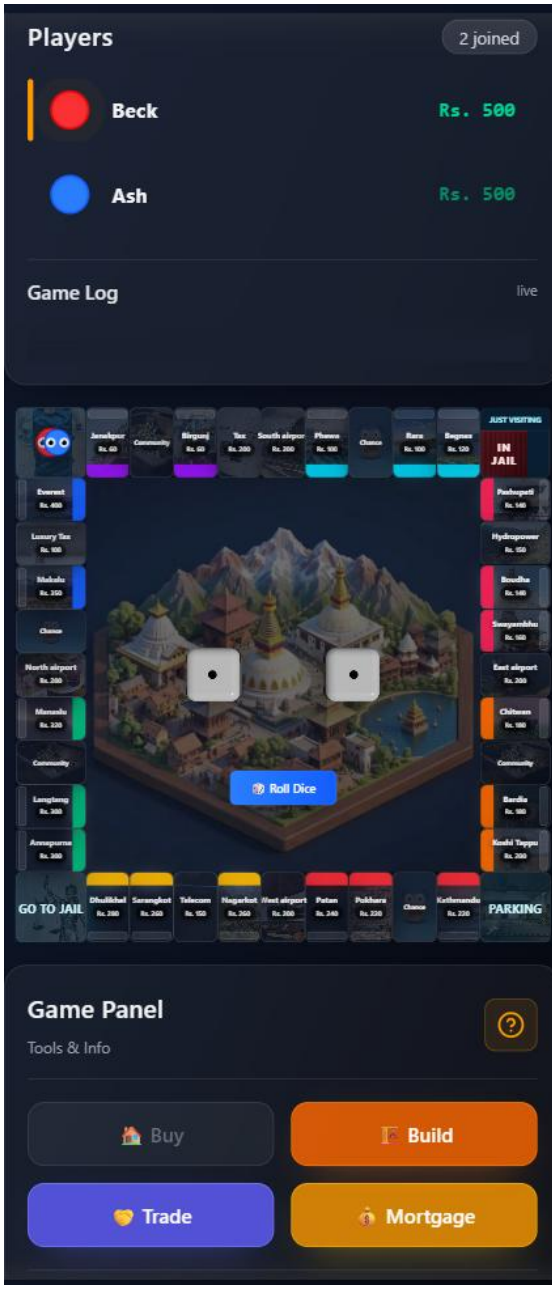
Annex 5: Trade UI of Monopoly Online



Annex 6: Rules Page of Monopoly Online



Annex 7: Lobby of Monopoly Online- Mobile View



Annex 8: Game Board of Monopoly Online - Mobile View

APPENDIX C: CODE SNIPPETS

```ts
import { DiceRoll } from "../types/game";
import { webcrypto } from "crypto";
export function rollDice(): DiceRoll {
  const values = new Uint32Array(2);
  webcrypto.getRandomValues(values);

  return {
    die1: (values[0]! % 6) + 1,
    die2: (values[1]! % 6) + 1,
    // die1: 6,
    // die2: 4,
  };
  // const results = [];
  // for (let i = 0; i < 1000; i++) {
  //   const roll = rollDice();
  //   results.push({ ...roll, total: roll.die1 + roll.die2 });
  // }
  // console.log(JSON.stringify(results, null, 2));
}
```

Annex 9: Dice Roll and RNG Fairness Test Code Snippet of Monopoly Online

```ts
export function startAuction(state: GameState, tile: PropertyTile): GameState {
  //start auction as someone declined to buy property
  const activePlayerIds = state.players
    .filter((p) => !p.isBankrupt)
    .map((p) => p.id);

  return {
    ...state,
    pendingAction: {
      type: "AUCTION",
      auction: {
        property: tile,
        highestBid: 0,
        activePlayerIds,
        currentBidderIndex: 0,
      },
    },
    events: [
      ...state.events,
      {
        type: "AUCTION_STARTED",
        property: tile,
      },
    ],
  };
}
```

Annex 10: Auction Logic Code Snippet of Monopoly Online

```ts
export function movePlayer(state: GameState, dice: DiceRoll): GameState {
  const player = getCurrentPlayerSafe(state);
  const steps = dice.die1 + dice.die2;

  const from = player.position;
  let to = from + steps;
  let passedGo = false;

  if (to >= BOARD_SIZE) {
    //TODO: passedGo,provide 200 as reward,but not for the jail crossing(i
    to = to % BOARD_SIZE;
    passedGo = true;
  }

  const updatedPlayer = {
    ...player,
    position: to,
    money: passedGo ? player.money + GO_REWARD : player.money,
  };

  const players = [...state.players];
  players[state.currentTurnIndex] = updatedPlayer;

  const events: GameEvent[] = [
    ...state.events,
    // { type: "DICE_ROLLED", dice },
    { type: "PLAYER_MOVED", from, to },
  ];

  if (passedGo) {
    events.push({ type: "PASSED_GO", amount: GO_REWARD });
  }

  return {
    ...state,
    players,
    lastDice: dice,
    events,
  };
}
```

Annex 11: Player Movement Code Snippet of Monopoly Online